DOCUMENTAZIONE TECNICA: Social Network for Music (SN4M)

Corso di Applicazioni Web e Cloud – A.A. 2024/2025

> Autore del progetto: Fokou Mychael
>
> Matricola: 29159A

SCOPO DEL PROGETTO

Il progetto consiste nello sviluppo di una piattaforma web denominata **SN4M (Social Network for Music)**, progettata per la gestione e la condivisione di playlist musicali tra utenti.

L'applicazione si appoggia alle **API di Spotify** per il recupero dei metadati (titoli, artisti, brani) e utilizza il **Web Storage** del browser per la persistenza dei dati.

STRUMENTI UTILIZZATI

Ambito	Tecnologia / Strumento
Estetica	HTML, CSS
Logica	JavaScript (Vanilla)
Framework UI	Bootstrap 5
Memorizzazione Dati	**Web Storage (LocalStorage)** Per salvare tutto sul browser senza bisogno di un server esterno.
Integrazione Dati	**Spotify Web API** per ottenere titoli, artisti e copertine reali.

DESCRIZIONE FILES

Il "Cervello" e la Struttura (Core Files)

Questi file lavorano "dietro le quinte" per far funzionare tutto il sito.

`App_Mike/auth.js`

- **Gestore della Sicurezza:** Controlla che nessuno entri nelle pagine protette senza login (reindirizza al login se non autenticato).
- **Gestore Sessione:** Gestisce le operazioni di Login e Logout.
- **Gestore Dati:** È il "magazziniere" centrale. Si occupa di salvare e recuperare i dati (sia privati dell'utente che pubblici delle community) in modo ordinato, separando i dati di un utente da quelli di un altro.

`App_Mike/layout.js`

- **Gestore Interfaccia Globale:** Costruisce automaticamente la **Sidebar** (menu laterale) e il **Player Musicale** in ogni pagina, evitando duplicazione di codice.
- **Connessione a Spotify:** Gestisce il "token" per comunicare con Spotify (lo richiede solo quando serve e lo rinnova se scaduto).
- **Riproduzione:** Gestisce la logica del player (Play/Pause, passaggio dei dati del brano cliccato).
- **Modale Salvataggio:** Gestisce la finestra per salvare un brano in una playlist.

`App_Mike/style.css`

- **Design Base:** Definisce i colori (sfondo nero/grigio), i font e la struttura (Sidebar fissa a sinistra, contenuto che scorre a destra) comune a tutto il sito.

Area Utente (Profilo e Accesso)

I file che gestiscono l'identità dell'utente.

`App_Mike/profilo/login.html`

- **Accesso:** Permette all'utente di inserire email e password. Verifica le credenziali tramite `auth.js`.

`App_Mike/profilo/register.html`

- **Registrazione:** Permette di creare un nuovo account. Include controlli di validità (email corretta, password sicura, username valido) e verifica unicità email.

`App_Mike/profilo/preferenze.html`

- **Onboarding:** Appare post-registrazione. Chiede all'utente i gusti iniziali (Genere e Artista) per personalizzare l'esperienza.

`App_Mike/profilo/profilo.html`

- **Gestione Account:** Visualizzazione e modifica dati (Username, Preferenze).
- **Cancellazione:** Funzione critica per eliminare definitivamente l'account e rimuovere i dati privati associati dal database.

`App_Mike/home2.html`

- **Homepage:** Pagina di atterraggio dopo il login.
- **Vetrina Musicale:** Mostra sezioni dinamiche ("Per te", "Consigliati", "Hit") recuperate da Spotify.
- **Ricerca Rapida:** Barra di ricerca che reindirizza alla pagina dedicata.

Gestione Musica e Playlist (Privato)

Dove l'utente gestisce la sua libreria personale.

`App_Mike/playlist/ricerca.html`

- **Motore di Ricerca:** Cerca brani o artisti tramite API Spotify.
- **Risultati:** Griglia di brani trovati. Cliccando su un brano, parte nel player e può essere salvato.

`App_Mike/playlist/playlist.html`

- **Editor Playlist:** Pagina di dettaglio playlist privata.

- **Gestione Contenuti:** Rinominare playlist, cambiare descrizione, rimuovere brani.
- **Gestione Tag:** Aggiungere/rimuovere etichette (Tag) per classificare la playlist.
- **Sincronizzazione Automatica:** Ogni modifica (titolo, tag, rimozione brani) viene "spinta" automaticamente anche nei post pubblici delle community.

Area Social (Community)

Dove avviene l'interazione tra utenti.

`App_Mike/community/community.html`

- **Elenco Gruppi:** Mostra tutte le community esistenti come card.
- **Creazione:** Permette di fondare nuove community (nome, descrizione, Tag).
- **Filtri:** Ricerca community per nome o filtrando per Tag specifici.
- **Badge Creatore:** Mostra visivamente quali community sono state create dall'utente corrente.

`App_Mike/community/x.community.html`

- **Bacheca (Feed):** Interno di una singola community. Mostra i post condivisi.
- **Condivisione:** Pubblicazione playlist privata nel feed.
- **Filtri Avanzati:** Ricerca nel feed per testo (titolo/brano) e per Tag contemporaneamente.
- **Interazioni:** Ascolto/importazione playlist altrui. Rimozione post (per creatore playlist). Modifica Tag community o eliminazione community (per creatore community).

WORKFLOW DI SVILUPPO

La web app **SN4M** è stata realizzata adottando una metodologia di sviluppo **"a strati"** (layered development). Invece di completare un singolo file alla volta, il progetto è avanzato per macro-funzionalità, attraversando ciclicamente tutti i file per ogni fase evolutiva.

> Vantaggio: Questo approccio ha permesso di testare immediatamente l'impatto di una nuova funzione (es. i Tag) su tutti i file correlati e mantenere coerenza stilistica e logica.

Il workflow è stato suddiviso nelle seguenti fasi operative:

1. **Strato del Design e Interfaccia (UI):** Definizione dell'aspetto estetico (CSS) per un'esperienza simile a Spotify e creazione scheletri HTML.
2. **Strato della Componentistica Globale:** Ottimizzazione elementi ricorrenti (Sidebar, Player) centralizzati in `layout.js` per iniezione dinamica.
3. **Strato dell'Identità e Sicurezza:** Implementazione sincronizzazione e memorizzazione credenziali. Logica `auth.js` per protezione rotte e persistenza dati.
4. **Strato dei Dati e Integrazione API:** Logica core: gestione playlist e integrazione API REST Spotify. Raffinamento sincronizzazione dati (modifiche private propagate ai feed pubblici).
5. **Strato di Ottimizzazione e Refactoring:** Pulizia codice, eliminazione ridondanze, ottimizzazione chiamate API (token centralizzato).
6. **Validazione dei Requisiti:** Check finale rigoroso sulle specifiche (Aggiunta/Modifica/Cancellazione playlist e community).

IMPLEMENTAZIONE E DETTAGLI TECNICI

Di seguito viene descritta l'implementazione tecnica divisa per strati, evidenziando i file coinvolti e le porzioni di codice fondamentali.

A. Strato del Design e Interfaccia (UI)

In questa fase è stata definita la struttura visiva dell'applicazione, garantendo un'esperienza utente coerente e responsive tramite l'uso di **Bootstrap 5** e un foglio di stile personalizzato.

File: `App_Mike/style.css`
Questo file funge da "gabbia" grafica per l'intera applicazione.

- **Struttura Layout:** Classe `.wrapper` con `display: flex` per affiancare sidebar fissa e contenuto scrollabile.
- **Scrollbar Personalizzata:** Stili `webkit` per coerenza con design dark.

```
/* Esempio di struttura layout in style.css */
.wrapper {
    display: flex;
    height: 100vh;
    padding-bottom: 90px; /* Spazio riservato al Player in basso */
}
.main-content {
    flex-grow: 1;
    background-color: #121212;
    overflow-y: auto; /* Scroll indipendente dal player */
}
```

B. Strato della Componentistica Globale

Per ottimizzare lo sviluppo e ridurre la ridondanza HTML, gli elementi ricorrenti sono stati centralizzati.

File: `App_Mike/layout.js`
Script motore dell'interfaccia globale. Inietta il DOM necessario al caricamento.

- **Rendering Sidebar:** `renderGlobalSidebar()` costruisce il menu ed evidenzia la pagina corrente (`window.location.pathname`). Popola dinamicamente la lista playlist.
- **Player Persistente:** `renderGlobalPlayer()` inietta il player. `updateGlobalPlayerUI()` gestisce lo stato senza interrompere la navigazione.

```
// Rendering dinamico della sidebar con stato attivo
function renderGlobalSidebar() {
    const sbContainer = document.getElementById('global-sidebar-container');
    const path = window.location.pathname;

    // Logica condizionale per evidenziare la pagina corrente
    const isHome = path.includes('home2.html');

    // ... iniezione HTML tramite Template Literals ...
    sbContainer.innerHTML = `...`;
    renderGlobalPlaylists(); // Popola la lista playlist personale
}
```

C. Strato dell'Identità e Sicurezza

Gestisce ciclo di vita utente, protezione rotte e isolamento dati.

File: `App_Mike/auth.js`
File centrale sicurezza.

- **Auth Guard:** Controlla presenza `utenteLoggato` in `sessionStorage`. Redirect se assente.
- **Isolamento Dati:** `saveUserData()` antepone l'email alle chiavi `localStorage` (es. `mario@mail.it_playlist`) per separare gli utenti.

```
// Auth Guard: Protezione delle rotte
(function checkAuth() {
    const user = sessionStorage.getItem("utenteLoggato");
    const path = window.location.pathname;
    const publicPages = ["login.html", "register.html"];

    // Se l'utente non c'è e la pagina non è pubblica → Redirect
    if (!user && !publicPages.some(page => path.includes(page))) {
        window.location.href = "/AWC/App_Mike/profilo/login.html";
    }
})();
```

File: `App_Mike/profilo/register.html`

- **Validazione Input:** Utilizzo Regex per regole sicurezza (password complessa).

```
// Validazione tramite Regex
const passwordRegex = /^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/;
if (!passwordRegex.test(password)) {
    alert("La password deve essere lunga almeno 8 caratteri e contenere almen
o un numero.");
    return;
}
```

D. Strato Gestione Dati e Integrazione API

Fase complessa di interazione con servizi esterni e logica business.

File: `App_Mike/layout.js` (Gestione Token)

- **Token Management:** `getAccessToken()` implementa *Client Credentials Flow*. Salva token in cache (`localStorage`) con timestamp per evitare rate-limiting.

```
window.getAccessToken = async function() {
    const now = Date.now();
    // Controllo se il token in cache è ancora valido
    if (storedToken && expiration && now < parseInt(expiration)) {
```

```
        return storedToken;
    }
    // Altrimenti, richiesta al proxy
    const response = await fetch("[https://accounts.spotify.com/api/token](http
s://accounts.spotify.com/api/token)...", {
        method: "POST",
        body: "grant_type=client_credentials"
    });
    // ... salvataggio token ...
}
```

File: `App_Mike/home2.html` (Data Fetching)

- **Popolamento Dinamico:** `loadSection()` usa `fetch` verso Spotify, parsifica JSON e genera card musicali.

```
async function loadSection(query, containerId) {
    const token = await window.getAccessToken();
    const response = await fetch(`https://api.spotify.com/v1/search?q=$...quer
y=${encodeURIComponent(query)}...`, {
        headers: { "Authorization": "Bearer " + token }
    });
    const data = await response.json();
    // Rendering dei risultati...
}
```

File: `App_Mike/playlist/playlist.html` (Logica Sincronizzazione)
Implementa il **Mirroring dei Dati**.

- **Sync:** `syncPublicFeeds(action)` garantisce coerenza privato/pubblico. Se l'utente modifica una playlist privata, la funzione aggiorna le copie nei feed community.

```
function syncPublicFeeds(action) {
    const communities = window.getPublicData('spotify_communities') || [];

    communities.forEach(comm => {
```

```
        // Logica di aggiornamento speculare
        if (action === 'update') {
            posts.forEach(p => {
                if (p.originalId === activePlaylist.id) {
                    p.title = activePlaylist.name; // Aggiorna Titolo
                    p.tags = activePlaylist.tags;  // Aggiorna Tag
                    p.tracks = activePlaylist.tracks; // Aggiorna Brani
                }
            });
        }
        // ... salvataggio modifiche nel feed pubblico ...
    });
}
```

File: `App_Mike/community/x.community.html` (Logica Sociale)
Gestione feed e filtri.

- **Filtro Avanzato:** `filtraFeed()` controlla contemporaneamente testo (titolo/brano) e Tag selezionati.

```
window.filtraFeed = function() {
    const q = document.getElementById('feedSearch').value.toLowerCase();
    const filtered = posts.filter(p => {
        // Match 1: Ricerca testuale nel titolo o nei brani
        const matchText = p.title.toLowerCase().includes(q) ||
                          p.tracks.some(t => t.title.toLowerCase().includes(q));
        // Match 2: Controllo incrociato dei Tag (Deve avere TUTTI i tag seleziona
ti)
        const matchTags = selectedFilterTags.length === 0 ||
            (p.tags && selectedFilterTags.every(tag => p.tags.includes(tag)));
        return matchText && matchTags;
    });
    renderFeed(filtered);
}
```

LISTA VARIABILI LOCALI

Mappa completa di tutto ciò che l'applicazione salva nella memoria del browser, divisa per tipologia.

- funzione si salvataggio: `sessionStorage.setItem` / `localStorage.setItem`

Local Storage (Dati Permanenti)

Database Master

- `utenti`: La lista completa di tutti gli account registrati (nomi, email, password e dati profilo)
  - Creato in: App_Mike/profilo/register.html
  - funzione register

Dati Privati Isolati

- `${email}_preferenze`: I gusti musicali specifici (genere e artista) dell'utente loggato.
  - Creato in: App_Mike/profilo/preferenze.html
  - funzione savePreferences
- `${email}_spotify_playlists`: Tutte le playlist create dall'utente (titoli, descrizioni, tag e brani contenuti)
  - Creato in: App_Mike/layout.js
  - funzione creaNuovaPlaylistGlobal

Dati Pubblici Globali

- `${email}_my_joined_communities`: L'elenco degli ID delle community a cui l'utente è iscritto.
  - Creato in: App_Mike/community/community.html
  - funzioni salvaNuovaCommunity
- `global_spotify_communities`: L'archivio di tutte le community create sulla piattaforma.
  - Creato in: App_Mike/community/community.html
  - funzione salvaNuovaCommunity

- `global_feed_{ID}`: Tutti i post e le playlist condivise all'interno di una specifica community.
  - Creato in: App_Mike/community/x.community.html
  - funzione apriModalCondivisione

Stato del Sistema

- `current_track`: I dettagli del brano attualmente caricato nel player (titolo, autore, immagine, durata).
  - Creato in: App_Mike/layout.js
  - funzione selezionaBrano
- `spotify_access_token`: La chiave di autorizzazione per poter interrogare i server di Spotify.
  - Creato in: App_Mike/layout.js
  - funzione window.getAccessToken
- `spotify_token_expiration`: L'orario preciso in cui il token di accesso scadrà.
  - Creato in: App_Mike/layout.js
  - funzione window.getAccessToken

NOTA: seppur rappresentano stati temporanei del sistema, queste informazioni vivono in localStorage per due motivi fondamentali di **User Experience (UX)** e **Ottimizzazione**:

1. **Persistenza al Refresh (Resilienza)**: se l'utente ricarica la pagina o chiude accidentalmente la scheda, il sistema deve poter recuperare istantaneamente l'ultimo brano in ascolto (`current_track`) tramite la funzione `restorePlayerState()`. Se fosse in *sessionStorage*, il player tornerebbe vuoto ad ogni caricamento.

2. **Efficienza delle API**: Il token di Spotify ha una durata di un'ora (`3600ms`). Salvandolo in `localStorage` insieme alla sua scadenza (`spotify_token_expiration`), l'applicazione può riutilizzarlo attraverso diverse sessioni di navigazione finché è valido, evitando di sovraccaricare il sistema con richieste di autorizzazione ogni volta che si riapre il browser.

Session Storage (Dati Temporanei)

- `utenteLoggato`: I dati dell'utente che ha effettuato l'accesso in questa sessione. Serve a far capire al sito "chi" sta navigando in questo momento.
  - **Creato in**: **App_Mike/auth.js**
  - funzione window.login

LOGICA DI SALVATAGGIO/AGGIORNAMENTO

Sebbene i dati siano gli stessi, vengono salvati in tre "luoghi" diversi per scopi differenti:

Database destinato al programmatore/amministratore

1. `sessionStorage` (Sessione Corrente):

- **Scopo**: Aggiorna l'identità dell'utente "in tempo reale".
- **Perché**: Serve a far sì che le altre pagine (come la Home o il Profilo) vedano immediatamente i nuovi gusti senza dover ricaricare tutto dal database globale. È una memoria volatile: se chiudi la scheda, questo dato sparisce.
- **funzioni di salvataggio e lettura:** `sessionStorage.setItem` e `sessionStorage.getItem`

2. `localStorage` - etichetta `utenti` (Database Globale):

- **Scopo**: Persistenza a lungo termine per il login.
- **Perché**: Qui modifichi l'anagrafica principale. Senza questo passaggio, al prossimo accesso (dopo un logout), il sistema ricaricherebbe i vecchi dati iniziali, perdendo le modifiche fatte in questa sessione.
- **funzioni di salvataggio e lettura:** `localStorage.setItem` e `localStorage.getItem`

Database destinato all'utente

3. `localStorage` - etichetta `${email}_key` (dati **Isolata per utente globali**):

- **Scopo**: Gestione Dati Privati Isolati.
- **Perché**: Questo garantisce che i gusti musicali, playlist...siano "blindati" in uno spazio dedicato, rendendo il recupero dei dati più veloce e sicuro rispetto alla scansione dell'intero array globale.
- **funzioni di salvataggio e lettura: dati isolati**: `saveUserData` e `getUserData` usano come prefisso l'email dell'utente loggato e come key l'etichetta del contenuto: `preferenze`, `spotify_playlists`...

4. `localStorage` **- etichetta** `${email}_community` (dati **Isolata per utente globali**):

- **Perché**: Questo garantisce che i dati di condivisizione nella sesione community siano resi pubblici indipendentemente dal creatore della community o playlist
- **funzioni di salvataggio e lettura: dati globali**: `savePublicData` e `getPublicData` usano come prefisso non un id ma la dicutura `global`